

01 Vectors

1.5 Unit Vector

Direction without magnitude — the bridge to polar and spherical coordinates

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>



What You Will Learn

1

Grasp the definition of a unit vector (direction vector): $\hat{\mathbf{a}} = \mathbf{a} / \|\mathbf{a}\|$, length 1, same direction as the original

3

Understand how the unit circle and unit sphere relate to vector length: what it means to land on, inside, or outside them

5

Master polar and spherical coordinates, and convert between them and Cartesian coordinates

2

Understand the "length $\times$ direction vector" decomposition $\mathbf{a} = \|\mathbf{a}\| \hat{\mathbf{a}}$ — a scalar for size, a unit vector for direction

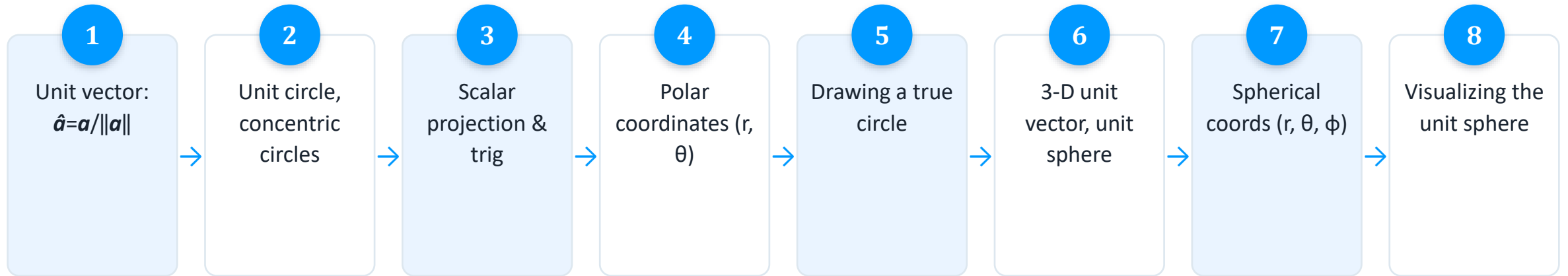
4

Master scalar projection and common trig functions, and how angle α relates to a unit vector's components

6

Compute and visualize unit vectors, unit circles, and unit spheres with NumPy and Matplotlib

How the Ideas Connect



Direction Vector: A Unit Vector Pointing the Same Way

$$\hat{\mathbf{a}} = \mathbf{a} / \|\mathbf{a}\|, \text{ where } \mathbf{a} \text{ is a nonzero vector}$$

- For any nonzero vector $\mathbf{a}$, dividing it by its own length $\|\mathbf{a}\|$ gives the direction vector $\hat{\mathbf{a}}$
- $\hat{\mathbf{a}}$ is also called a unit vector or normalized vector; "direction vector" is used when emphasizing direction
- $\hat{\mathbf{a}}$ points the same way as $\mathbf{a}$, and always has length 1, i.e. $\|\hat{\mathbf{a}}\|=1$
- Note: the zero vector $\mathbf{0}$ is usually considered to have no direction — its magnitude is zero, so it cannot be normalized

2-D Unit Vector: Formula and Example

$$\hat{\mathbf{a}} = \mathbf{a} / \|\mathbf{a}\| = [a_1 / \sqrt{a_1^2 + a_2^2}; a_2 / \sqrt{a_1^2 + a_2^2}]$$

- For any nonzero 2-D vector $\mathbf{a} = [a_1, a_2]^T$, divide each component by the length
- Example: $\mathbf{a} = [3, 4]^T$ has length $\sqrt{3^2 + 4^2} = 5$, so its unit vector is $\hat{\mathbf{a}} = [3/5, 4/5]^T$
- Normalizing only changes the length (to 1) — never the direction; geometrically it is a pure rescaling

Figure 1 · Normalizing a 2-D Vector $\mathbf{a}$

- The blue arrow is the original vector $\mathbf{a}=[3,4]^T$; the green arrow is its direction vector $\hat{\mathbf{a}}=[3/5,4/5]^T$
- The tip of $\hat{\mathbf{a}}$ lands exactly on the unit circle (dashed) centered at the origin — its length is 1

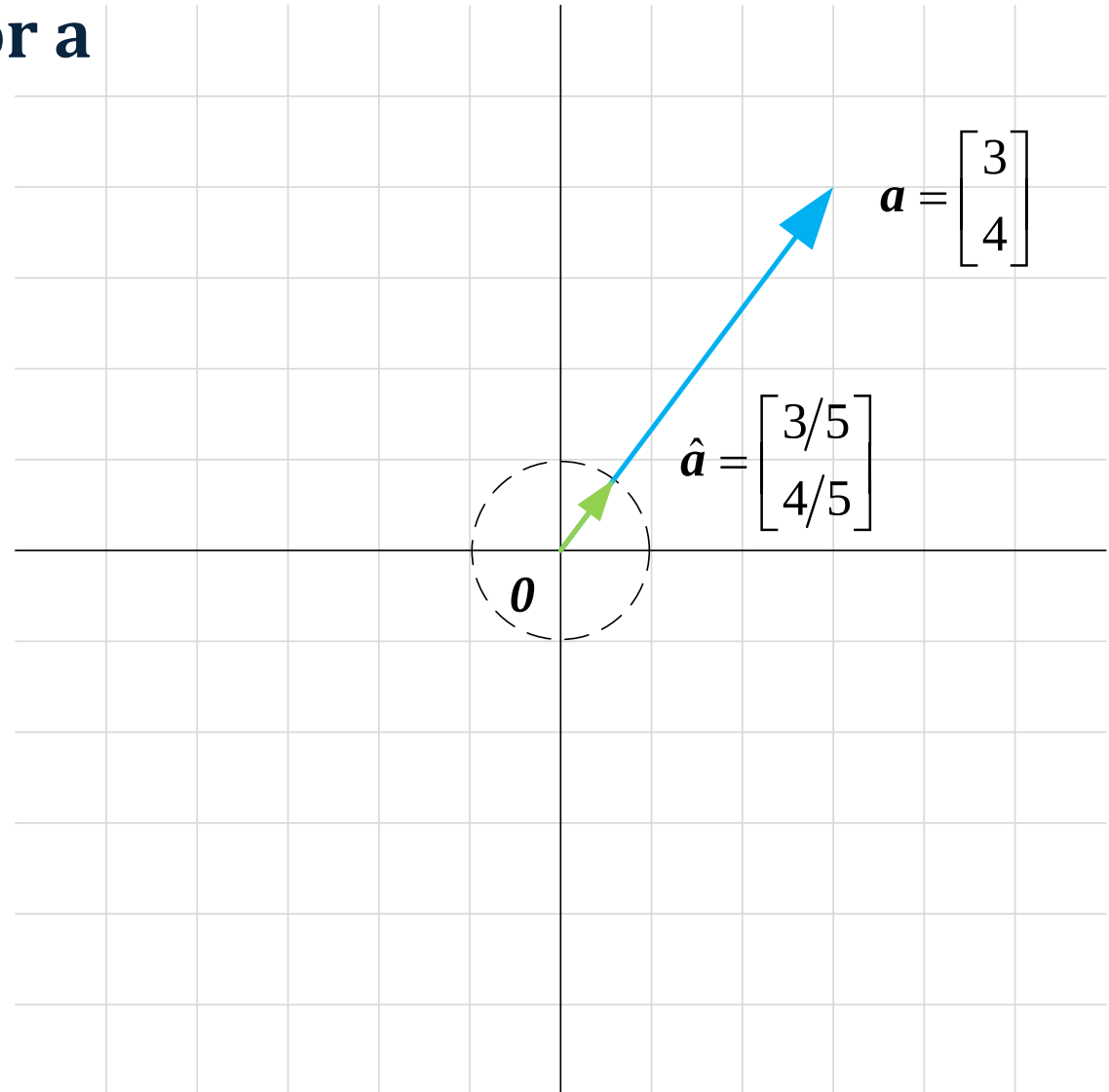
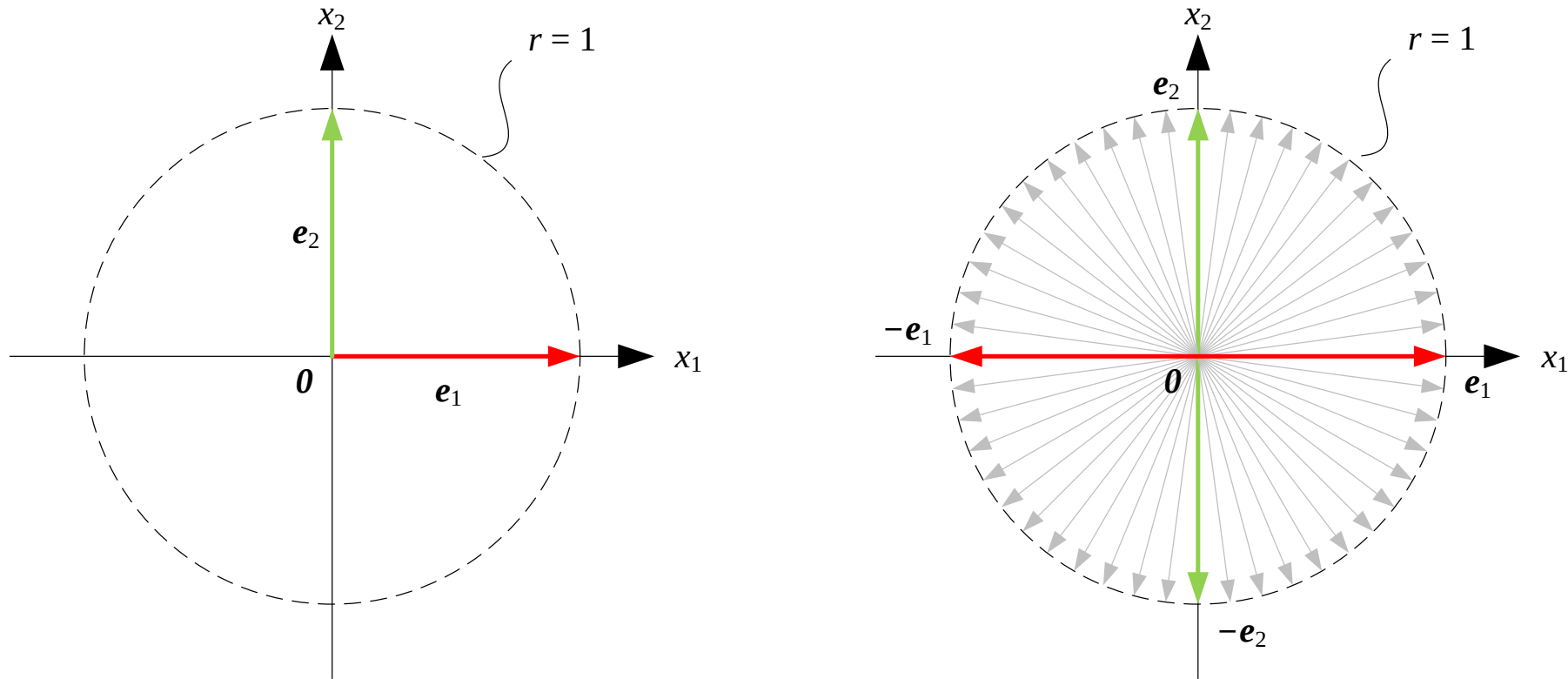


Figure 2 · Unit Vectors in the Plane: Origin to the Unit Circle

- $\mathbf{e}_1 = [1, 0]^T$ and $\mathbf{e}_2 = [0, 1]^T$ mark the positive x_1 , x_2 directions; $-\mathbf{e}_1$, $-\mathbf{e}_2$ mark the negative ones
- Like every other unit vector, these four special ones all have their tips on the unit circle



Length × Direction Vector: Another Way to Decompose a Vector

$\mathbf{a} = \|\mathbf{a}\| \hat{\mathbf{a}}$, where $\|\mathbf{a}\|$ is the length and $\hat{\mathbf{a}}$ is the direction vector

- $\|\mathbf{a}\|$ supplies the magnitude (length, L2 norm, Euclidean norm); $\hat{\mathbf{a}}$ supplies the direction
- Example: $\mathbf{a}=[3,4]^T$ can be written $5 \times [3/5, 4/5]^T$, where 5 is the length and $[3/5, 4/5]^T$ is the direction vector
- This decomposition cleanly separates "size" from "direction" — the foundation for projection and coordinate changes

Computing a Unit Vector with NumPy

np.linalg.norm() gives the length; a / norm_a normalizes it

- Define the vector $a = [3, 4]$ with `np.array()`
- Compute its length `norm_a` with `np.linalg.norm(a)`
- `unit_a = a / norm_a` normalizes it, giving the direction vector
- `norm_a * unit_a` restores "length $\times$ direction vector", checking it equals the original a

```
## 初始化
import numpy as np

## 定义向量 a
a = np.array([3, 4])

## 计算向量的长度
norm_a = np.linalg.norm(a)

## 计算单位向量
unit_a = a / norm_a

## 长度  $\times$  方向向量
norm_a * unit_a
```



Unit Circle, Concentric Circles: Position Set by Vector Length

$\|a\| < 1 \rightarrow$ inside the circle; $\|a\| = 1 \rightarrow$ on the circle; $\|a\| > 1 \rightarrow$ outside the circle

- With its tail at the origin, whether a vector's tip lands on the unit circle depends only on whether its length equals 1
- All vectors of the same length r starting at the origin have tips on a circle of radius r
- Different lengths give circles of different radii — together they form a family of circles concentric at the origin

Figure 3 · Vector Tips Inside, On, and Outside the Unit Circle

- Blue vectors have length under 1, landing inside the unit circle;
- red vectors have length over 1, landing outside
- Gray vectors have length exactly 1, landing on the unit circle (dashed)

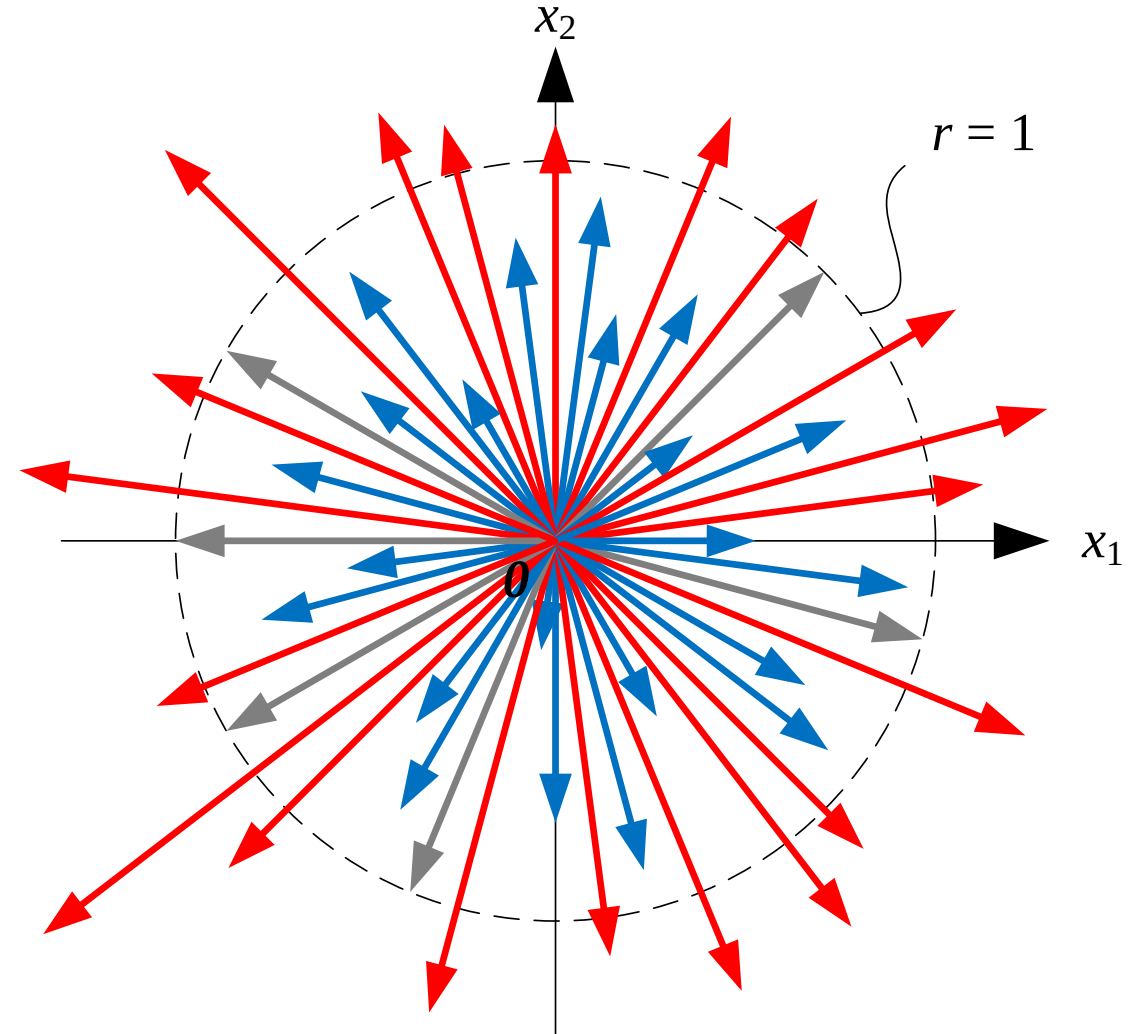
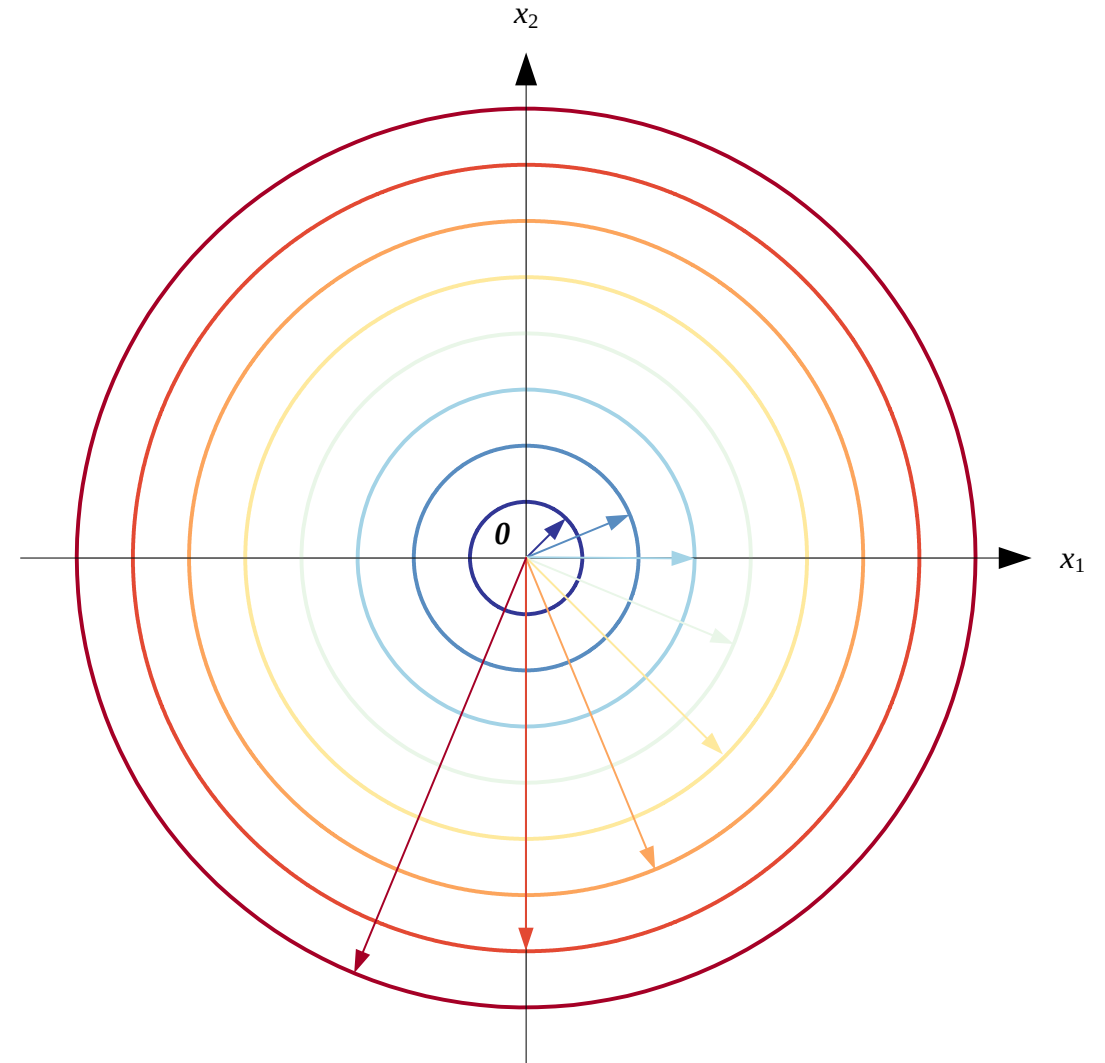


Figure 4 · Concentric Circles from Different Vector Lengths

- Vectors from the origin with the same length trace out one circle; different lengths give circles of different radii
- The radii of this family are exactly the vectors' L2 norms — explored further in the upcoming section on vector norms



Coordinates Are Projections: Both Axes Are Scalar Projections

$$a_1 = \|\mathbf{a}\| \cos \alpha, a_2 = \|\mathbf{a}\| \sin \alpha$$

- In Cartesian coordinates, the x-coordinate a_1 of $\mathbf{a}=[a_1, a_2]^T$ is its orthogonal projection onto the horizontal axis
- The y-coordinate a_2 is its orthogonal projection onto the vertical axis — both are "scalar projections"
- A scalar projection is the (signed) length of one vector's projection onto another direction, covered later in this chapter

Figure 5 · Scalar Projections of a Vector onto the Axes

- α is the angle between $\mathbf{a}$ and the positive horizontal direction;
the Pythagorean theorem gives $a_1 = \|\mathbf{a}\| \cos \alpha$, $a_2 = \|\mathbf{a}\| \sin \alpha$
- Like the noon sun shining straight down, a pole's shadow length is exactly its orthogonal projection onto the ground

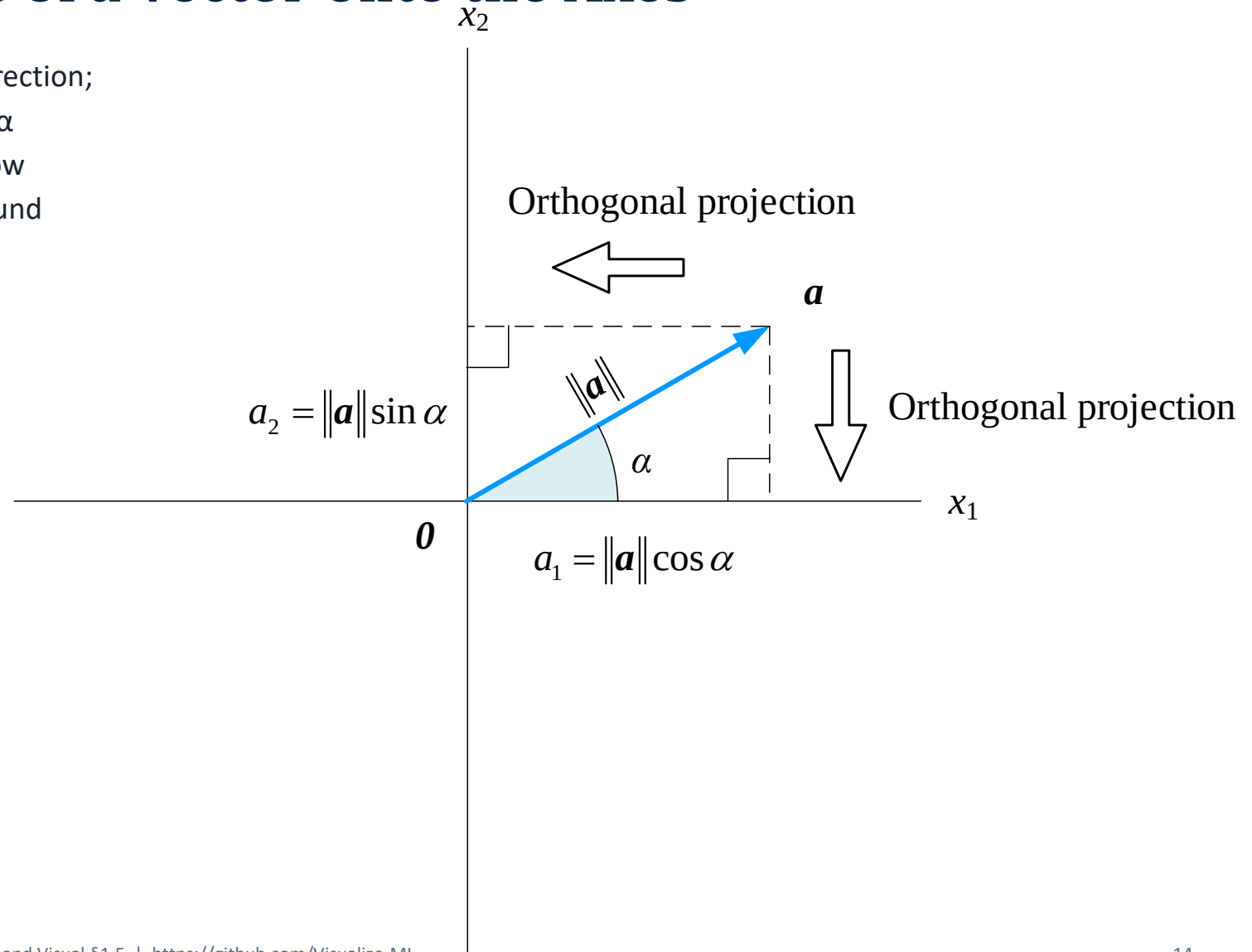
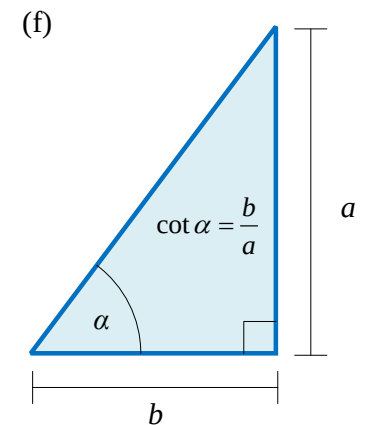
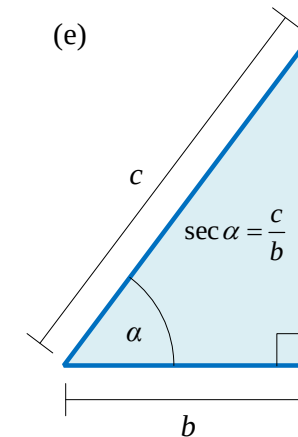
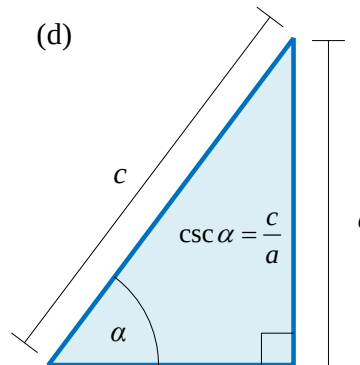
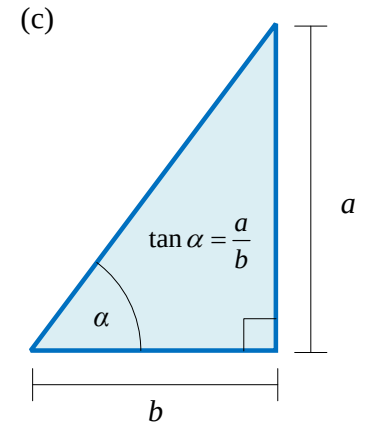
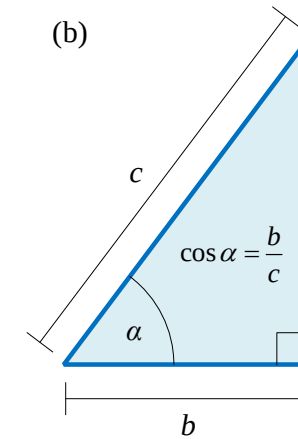
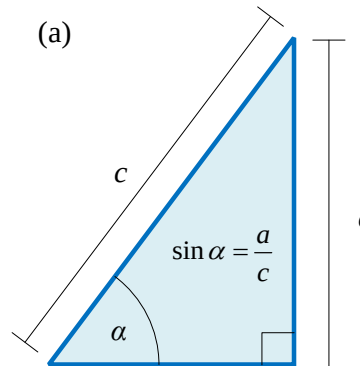


Figure 6 · Common Trig Functions (Sine, Cosine, Tangent, etc.)

- (a)(b)(c) $\sin \alpha = \text{opposite/hypotenuse}$, $\cos \alpha = \text{adjacent/hypotenuse}$, $\tan \alpha = \text{opposite/adjacent}$
- (d)(e)(f) cosecant $\csc \alpha$, secant $\sec \alpha$, and cotangent $\cot \alpha$ are the reciprocals of sine, cosine, and tangent



Length × Direction Vector: Direction Expressed via Angle α

$$\alpha = \arctan(a_2/a_1), \mathbf{a} = \|\mathbf{a}\| [\cos \alpha; \sin \alpha]$$

- α , the angle between $\mathbf{a}$ and the positive horizontal direction, comes from the arctangent function
- $[\cos \alpha; \sin \alpha]^T$ is exactly the direction vector $\hat{\mathbf{a}}$ of $\mathbf{a}$, always of length 1
- This form splits a vector cleanly into two scalars — "length" and "angle" — setting up polar coordinates

The Polar Coordinate System: Locating Points by Length and Angle

Pole The origin of the polar coordinate system

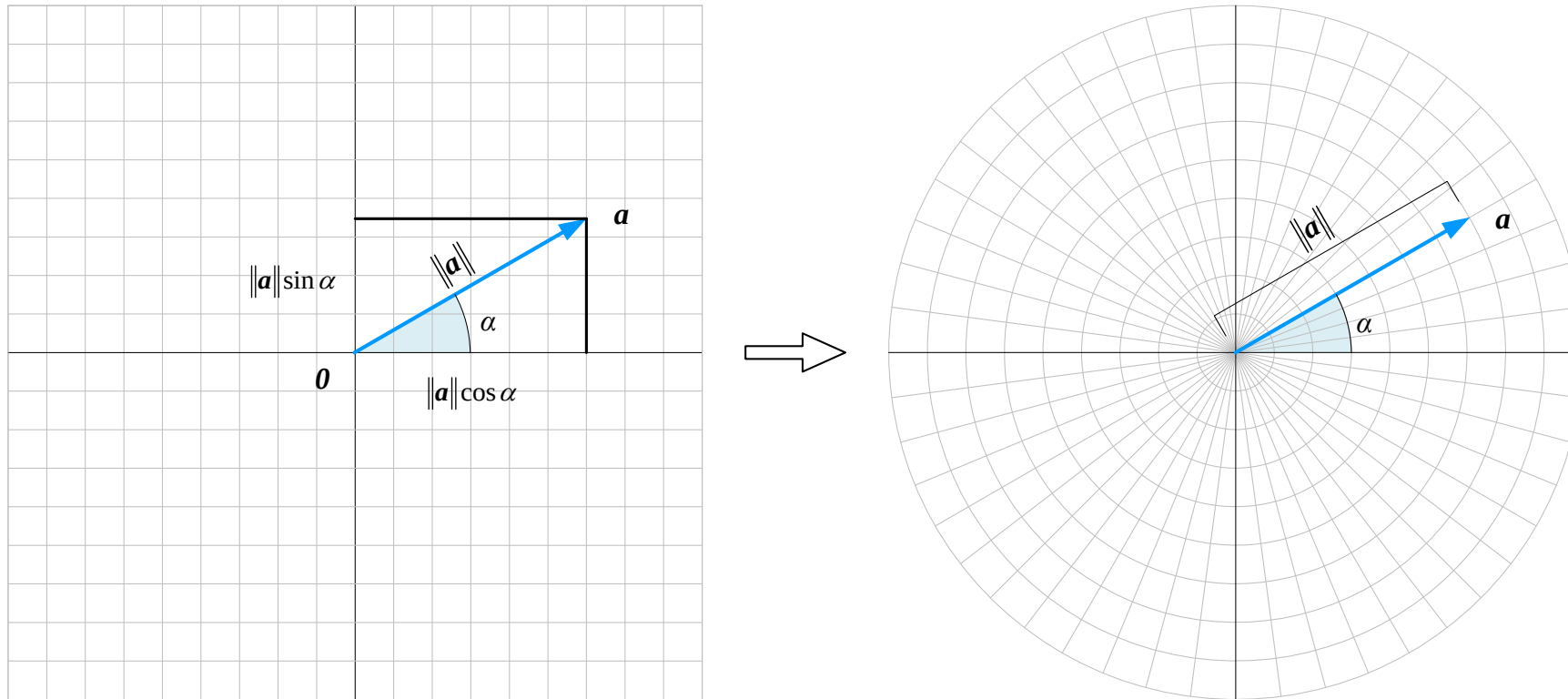
Polar Axis A reference ray from the pole, usually aligned with the positive horizontal axis

Radial Distance r The distance from a point to the pole; for vector $\mathbf{a}$, r corresponds to $\|\mathbf{a}\|$

Polar Angle θ The angle between the radius and the polar axis, positive counterclockwise, in $[0, 2\pi)$

Figure 7 · From the Cartesian Plane to Polar Coordinates

- The same vector $\mathbf{a}$ can be described in Cartesian coordinates (a_1, a_2) or in polar coordinates $(\|\mathbf{a}\|, \alpha)$
- The two are equivalent — just different languages for describing "position": axes-and-coordinates versus length-and-angle



Converting Between Cartesian and Polar Coordinates

np.arctan2() correctly handles angles in every quadrant

- `r = np.sqrt(x1**2 + x2**2)` computes the radial distance — the vector's length
- `theta = np.arctan2(x2, x1)` computes the polar angle without any extra quadrant checks
- `np.rad2deg(theta)` converts radians to degrees for easier interpretation
- `x1_ = r*np.cos(theta)`, `x2_ = r*np.sin(theta)` convert polar coordinates back to Cartesian

```
## 初始化
import numpy as np

## 平面直角坐标转化为极坐标
x1, x2 = 3, 4

# 计算极径
a r = np.sqrt(x1**2 + x2**2)

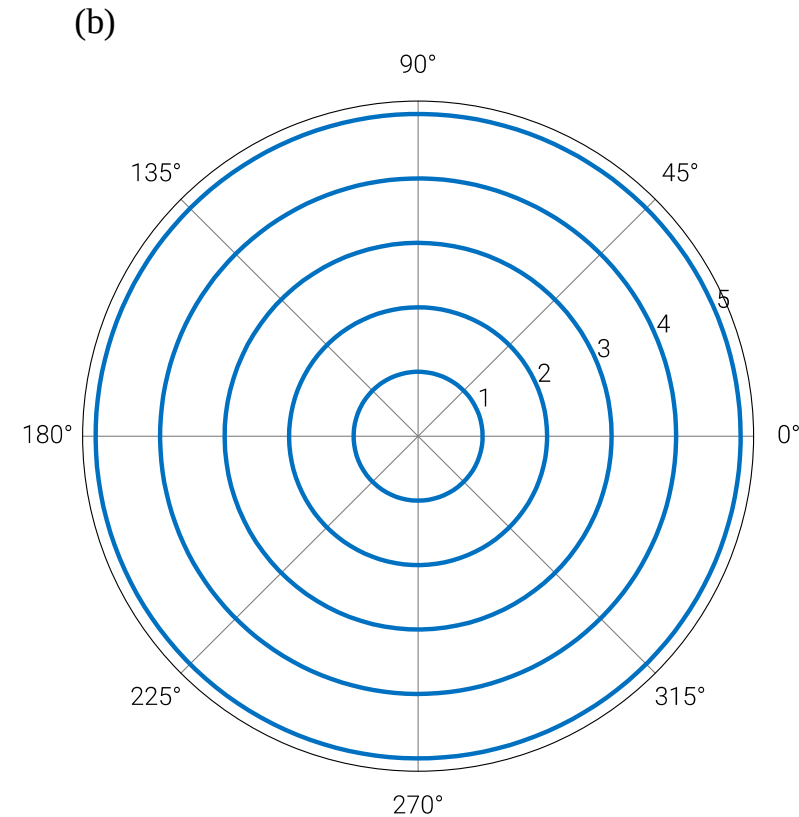
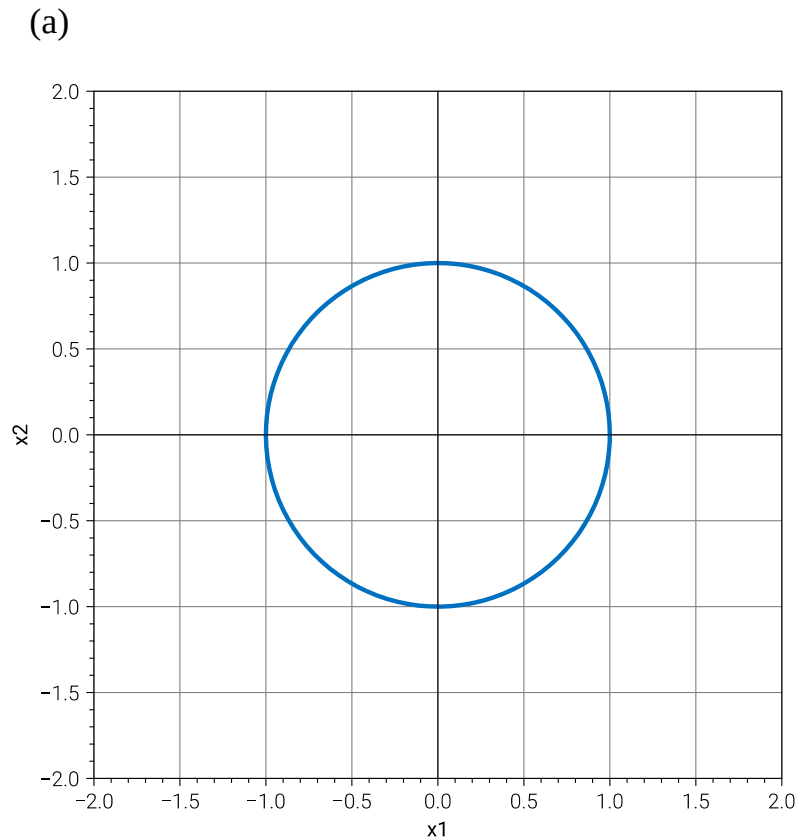
# 计算极角
b theta = np.arctan2(x2, x1)
c np.rad2deg(theta)

## 极坐标转化为平面直角坐标
x1_ = r * np.cos(theta)
x2_ = r * np.sin(theta)
```



Figure 8 · Visualizing a Circle: Cartesian and Polar Views

- (a) In Cartesian coordinates, 300 evenly-angled points trace the unit circle; equal axis scaling keeps it from looking squashed
- (b) In polar coordinates, several concentric circles of different radii are drawn, with angle ticks evenly marked 0° – 360°



Drawing a Unit Circle in Cartesian Coordinates

ax.plot() connects 300 points in sequence to outline the circle

- `theta_array` uses `np.linspace(0, 2 $\pi$ , 300)` to sample 300 evenly spaced angles around a full turn
- `x1_array=cos(theta_array)`, `x2_array=sin(theta_array)` give the matching points on the unit circle
- `ax.set_aspect('equal')` gives the axes equal unit length, so the circle isn't shown as an ellipse
- `ax.axhline()/ax.axvline()` draw the axes; `ax.set_xlabel()/set_ylabel()` add axis labels

```
## 初始化
import numpy as np
import matplotlib.pyplot as plt

## 生成数据
a theta_array = np.linspace(0, 2 * np.pi, 300) # 0 到 2 $\pi$  之间均匀取点
x1_array = np.cos(theta_array)
x2_array = np.sin(theta_array)

## 可视化
b fig, ax = plt.subplots(figsize=(6,6))

# 绘制单位圆
c ax.plot(x1_array, x2_array, 'k', linewidth=1) # 黑色圆, 线宽 1

# 设置 XY 轴比例相同
d ax.set_aspect('equal')
ax.grid(True)

# 设置坐标轴范围
ax.set_xlim([-2, 2])
ax.set_ylim([-2, 2])

# 添加坐标轴
ax.axhline(0, color='k', linewidth=0.5)
ax.axvline(0, color='k', linewidth=0.5)

# 增加轴标签
ax.set_xlabel('x1')
ax.set_ylabel('x2')
```



3-D Unit Vector: Formula and Example

the unit vector of $\mathbf{b}=[2,3,6]^T$ is $\mathbf{b}/\|\mathbf{b}\| = [2/7, 3/7, 6/7]^T$

- For any nonzero 3-D vector $\mathbf{a}=[a_1,a_2,a_3]^T$, its unit vector divides each component by $\sqrt{a_1^2+a_2^2+a_3^2}$
- Example: $\mathbf{b}=[2,3,6]^T$ has length $\sqrt{2^2+3^2+6^2}=7$, giving unit vector $[2/7,3/7,6/7]^T$
- Equivalently, "length $\times$ direction vector": $\mathbf{b} = 7 \times [2/7,3/7,6/7]^T$

Figure 9 · 3-D Unit Vectors End on the Unit Sphere

- Left: $\mathbf{e}_1$, $\mathbf{e}_2$, $\mathbf{e}_3$ are the unit vectors along the positive x_1 , x_2 , x_3 axes — all with tips on the sphere
- Right: countless unit vectors in every direction densely cover the sphere's surface, illustrating the "unit sphere"

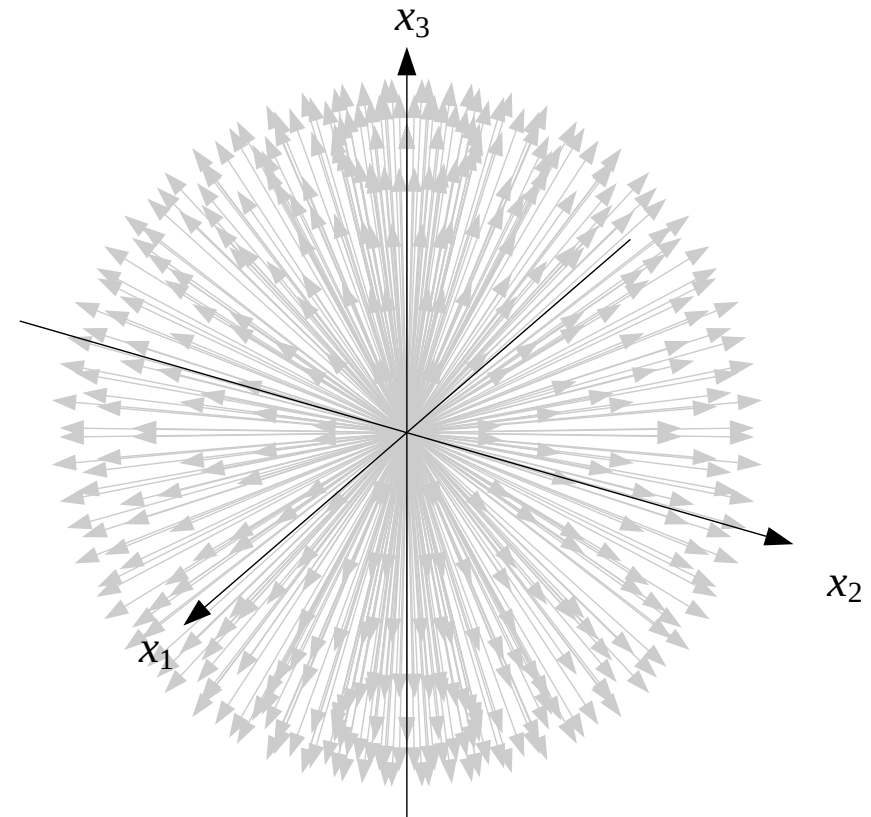
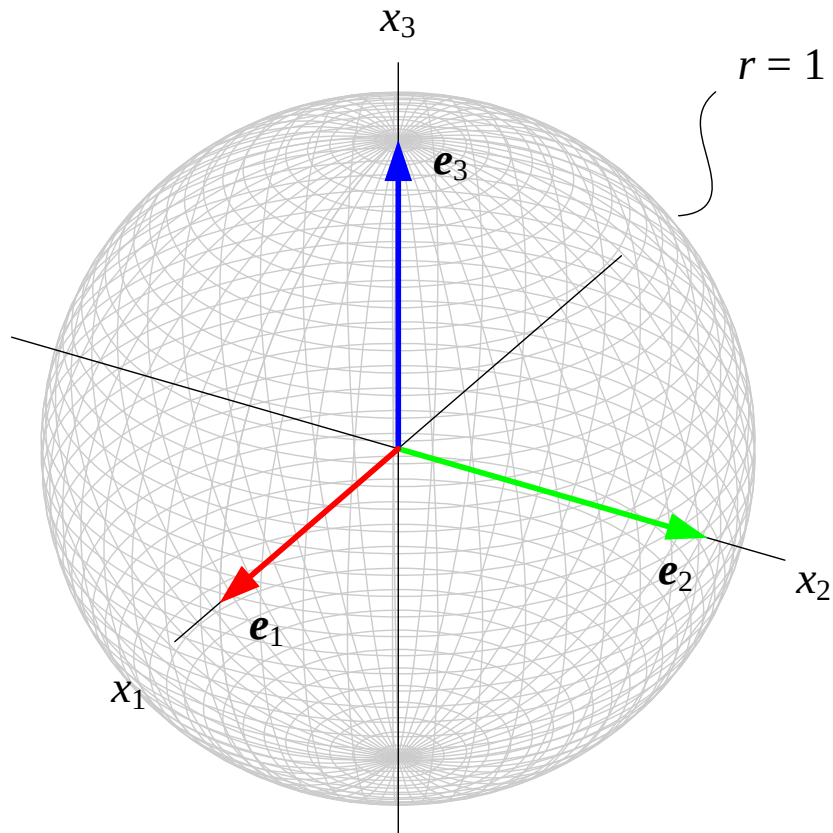
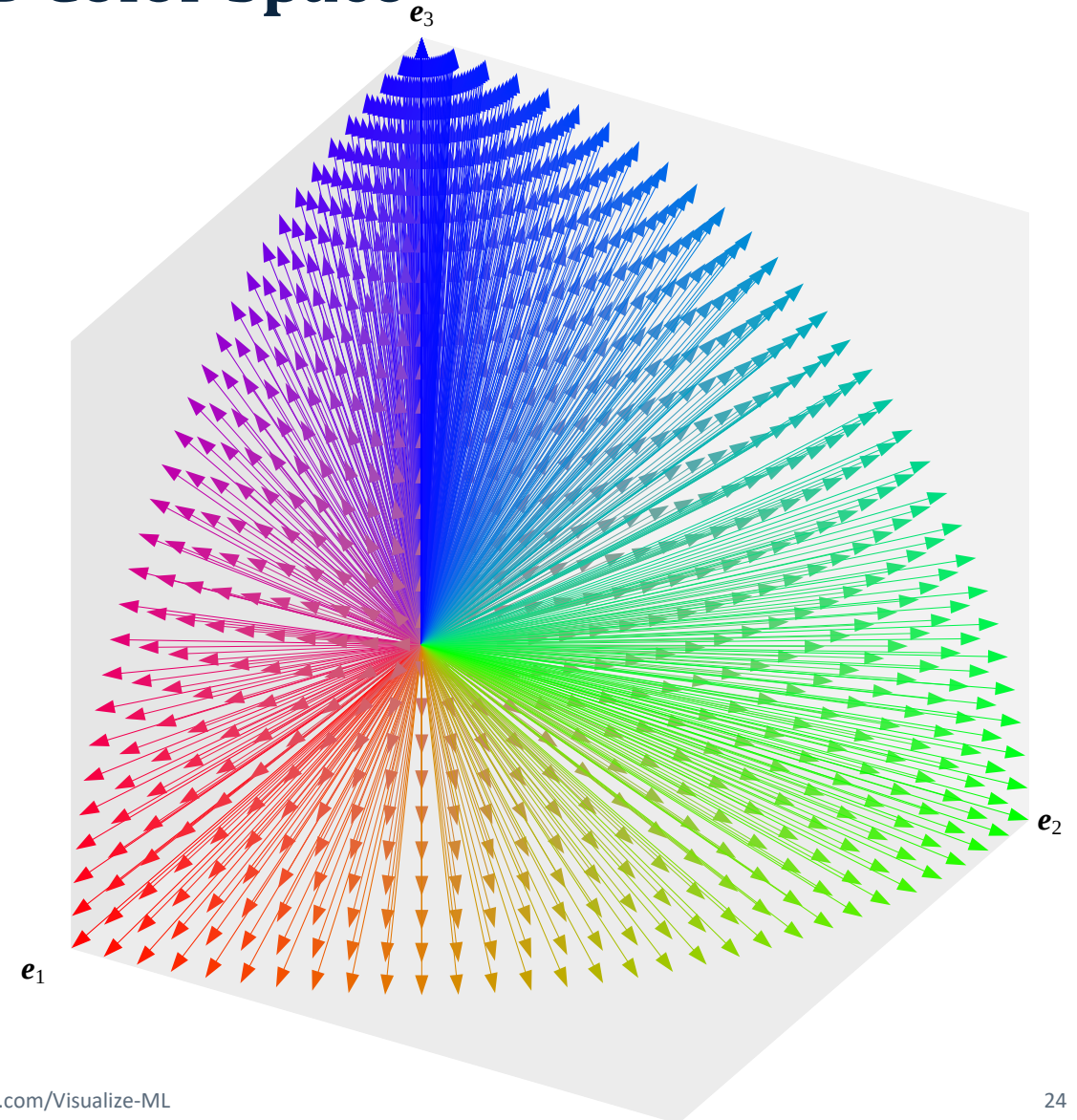


Figure 10 · Unit-Length Vectors in RGB Color Space

- The red, green, blue basis vectors e_1 , e_2 , e_3 and many vectors between them have tips on $1/8$ of the unit sphere
- Note $[1,1,1]^T$ has length $\sqrt{3}$, beyond the unit sphere — the white vector lies outside it, not on its surface



The Spherical Coordinate System: Built from Two Polar Systems

Radial Distance r The distance from point P to the origin O; for a vector this is $\|\mathbf{a}\|$

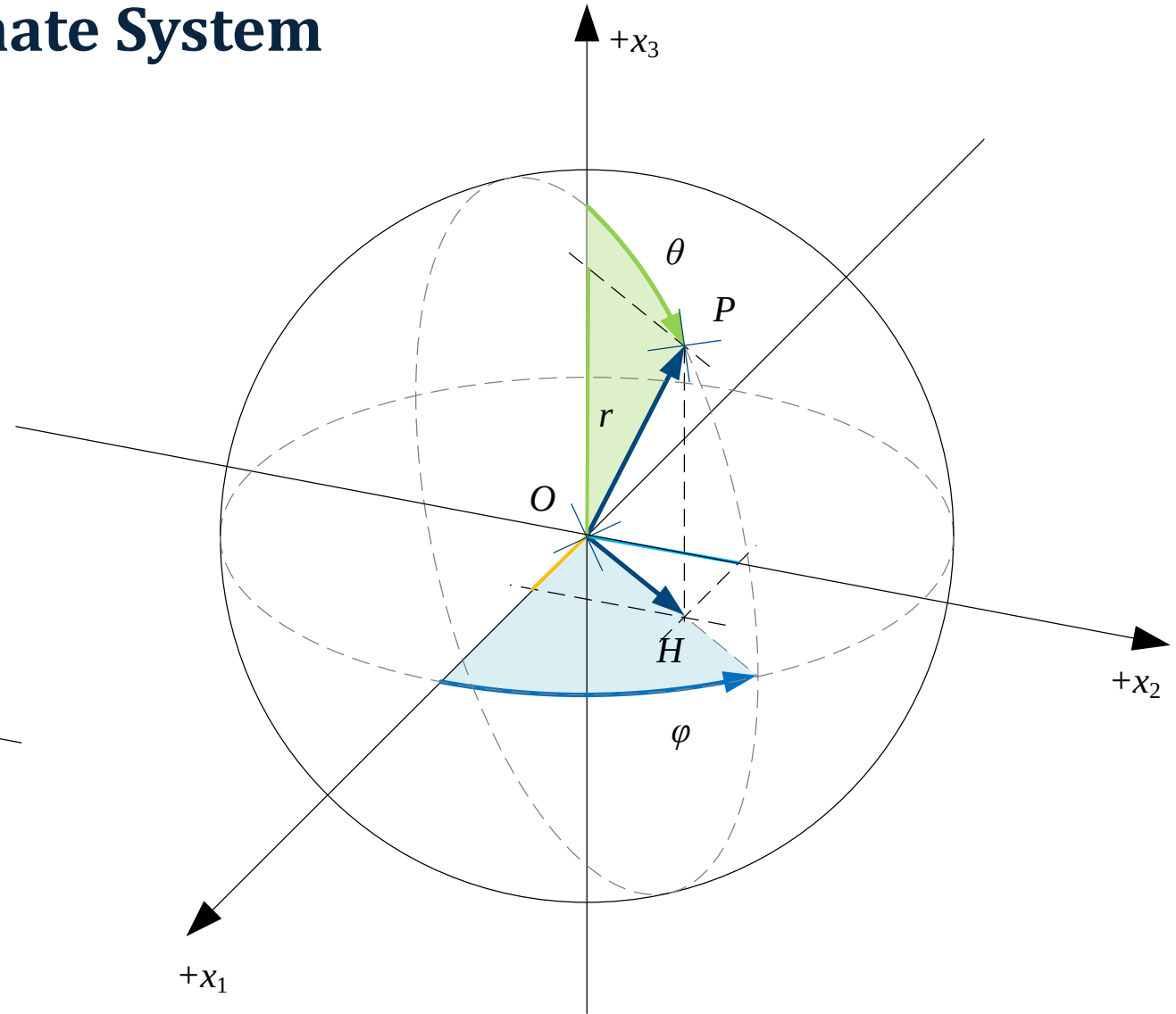
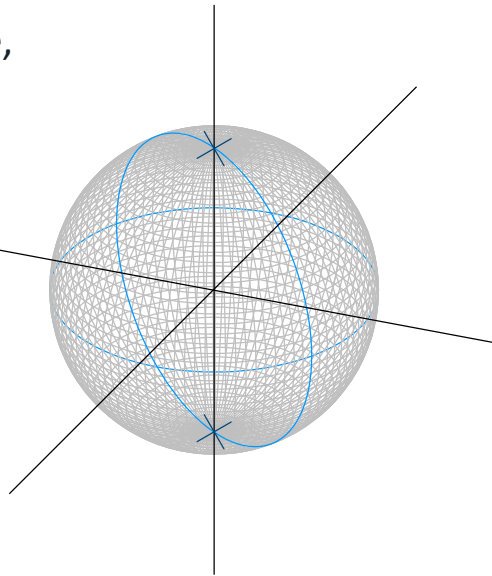
Polar Angle θ The angle between OP and the positive x_3 axis, in $[0, \pi]$ — like Earth's "latitude"

Azimuth Angle ϕ The angle between OP's projection onto the x_1x_2 plane and the x_1 axis, in $[0, 2\pi]$ — like "longitude"

Spherical (r, θ, ϕ) These three numbers uniquely fix a point P in 3-D space

Figure 11 · The Spherical Coordinate System

- Spherical coordinates are built from two planar polar systems — one fixing r and θ , the other fixing the projection direction and ϕ
- Point P 's spherical coordinates are written (r, θ, ϕ) — radial distance, polar angle, and azimuth angle
- Image from The Beauty of Visualization (《可视之美》)



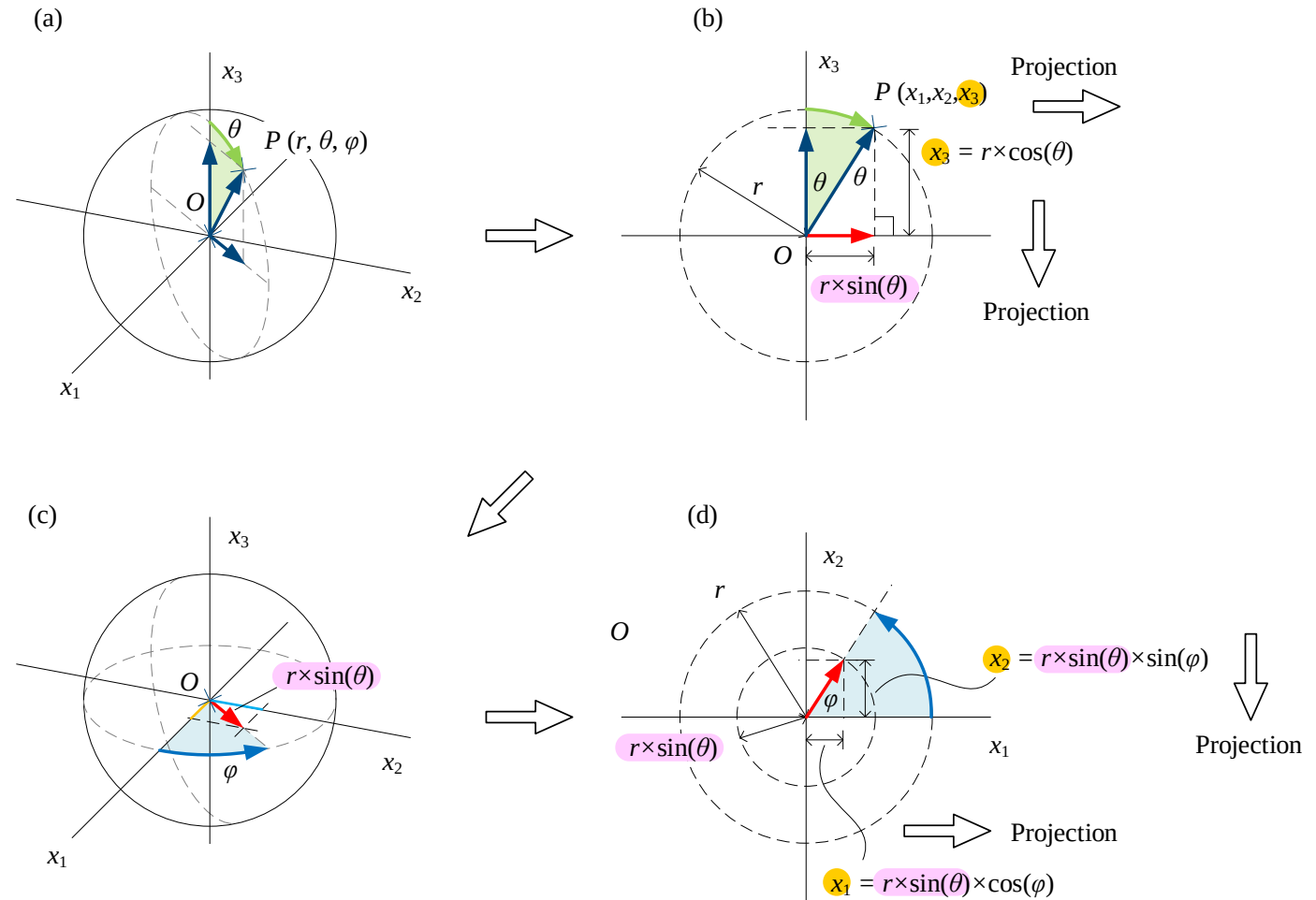
Step One: Projecting onto the x_3 Axis and the Horizontal Plane

$x_3 = r \cos \theta$; OP 's projection onto the x_1x_2 plane has length $r \sin \theta$

- Since x_3 is perpendicular to the x_1x_2 plane, r 's projection onto x_3 is exactly P 's x_3 -coordinate
- PO 's projection onto the x_1x_2 plane has length $r \sin \theta$, which must still be split into x_1 and x_2 directions

Figure 12 · From Spherical to 3-D Cartesian Coordinates

- (a)(b) projecting $r \sin \theta$ onto x_3 gives $x_3 = r \cos \theta$; (c) $r \sin \theta$ is OP's projection onto the x_1x_2 plane
- (d) projecting $r \sin \theta$ further onto x_1, x_2 gives $x_1 = r \sin \theta \cos \phi$ and $x_2 = r \sin \theta \sin \phi$



The Complete Conversion Between Spherical and 3-D Cartesian Coordinates

$$x_1 = r \sin\theta \cos\varphi, x_2 = r \sin\theta \sin\varphi, x_3 = r \cos\theta; \text{ conversely } r = \sqrt{x_1^2 + x_2^2 + x_3^2}$$

- Combining the three equations, point P corresponds to $r \times [\sin\theta \cos\varphi; \sin\theta \sin\varphi; \cos\theta]$, the bracket being the direction vector
- Going the other way, the Pythagorean theorem recovers r from Cartesian coordinates — this is the vector's length

Plotting a Unit Sphere's Wireframe

plot_wireframe() draws a 3-D mesh surface

- `np.linspace()` generates longitude u (0 to 2π) and latitude v (0 to π); `np.meshgrid()` builds the grid
- $xx=\cos(u)\sin(v)$, $yy=\sin(u)\sin(v)$, $zz=\cos(v)$ give the 3-D coordinates of each grid point on the sphere
- `ax.plot_wireframe(xx, yy, zz)` draws the mesh lines; `ax.set_aspect('equal','box')` keeps all three axes to scale
- `ax.set_proj_type('ortho')` uses orthographic projection so the sphere isn't distorted by perspective

```
## 初始化
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.cm import hsv

## 创建球面数据
u = np.linspace(0, 2 * np.pi, 37) # 生成经度角
v = np.linspace(0, np.pi, 19)      # 生成纬度角
uu, vv = np.meshgrid(u, v)         # 生成网格
r = 1 # 球半径

# 计算球面坐标
xx = r*np.cos(uu) * np.sin(vv)
yy = r*np.sin(uu) * np.sin(vv)
zz = r*np.cos(vv)

## 可视化
fig = plt.figure(figsize=(6, 6))
ax = fig.add_subplot(111, projection='3d')

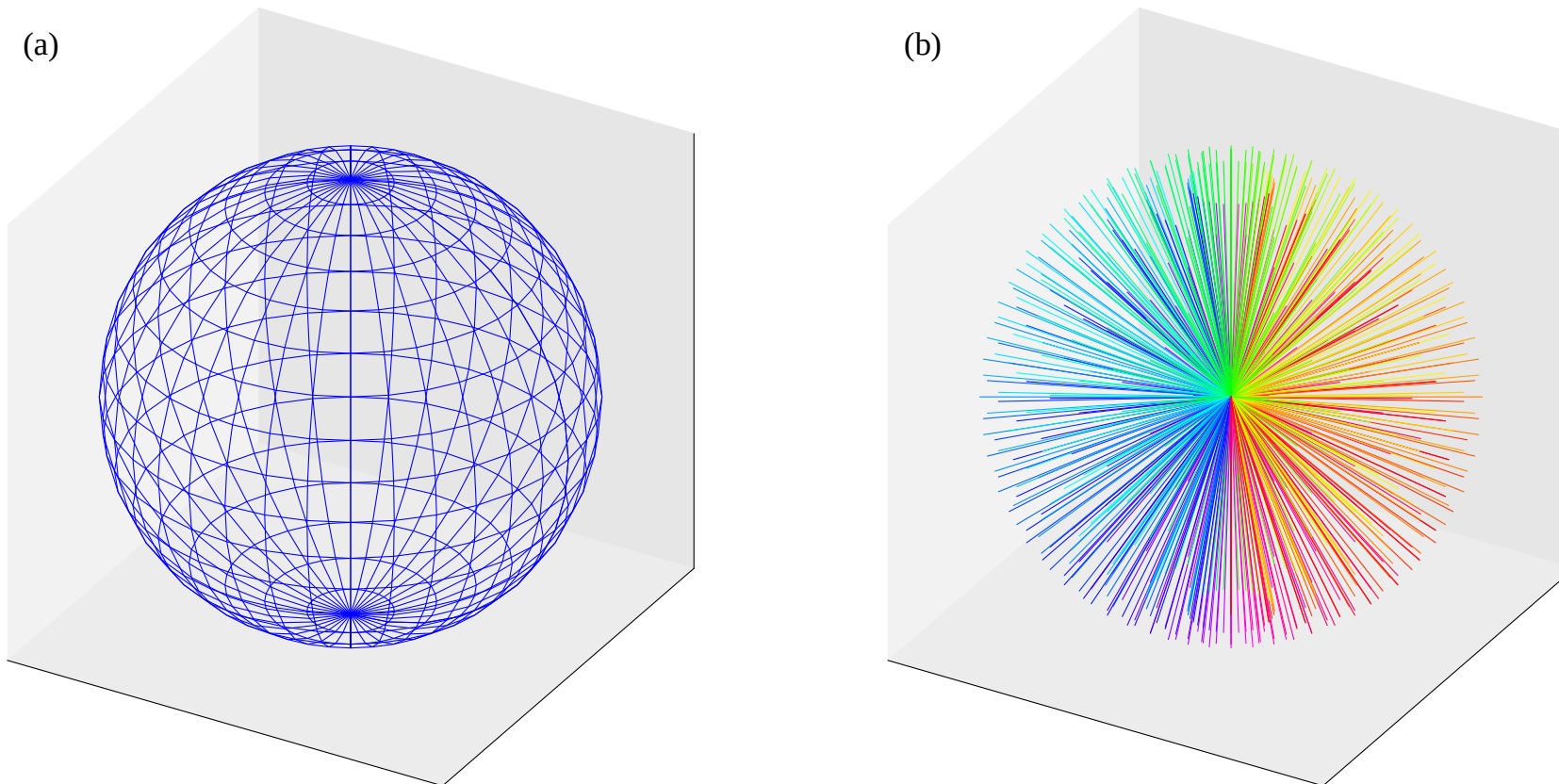
# 绘制球面网格线
ax.plot_wireframe(xx, yy, zz, color='blue', linewidth=0.5)

# 设置坐标轴范围
ax.set_xlim([-1, 1])
ax.set_ylim([-1, 1])
ax.set_zlim([-1, 1])

# 隐藏坐标轴刻度
ax.set_xticks([])
ax.set_yticks([])
ax.set_zticks([])
ax.set_aspect('equal', 'box')
ax.set_proj_type('ortho')
```

Figure 13 · Visualizing the Unit Sphere

- (a) A blue wireframe traces the unit sphere's surface — every point exactly distance 1 from the origin
- (b) Colorful unit vectors densely covering the sphere further show how 3-D unit-vector tips fill the whole surface



Key Terms in This Section

Unit Vector $\hat{a} = \mathbf{a} / \|\mathbf{a}\|$, length 1

Length $\times$ Direction $\mathbf{a} = \|\mathbf{a}\| \hat{a}$, decoupling size from direction

Scalar Projection the projected length of a vector onto some direction

Radial Distance / Polar Angle the two components of polar coordinates

Azimuth Angle φ — the angle between OP's x_1x_2 -plane projection and the x_1 axis

Direction Vector a unit vector pointing the same way as the original

Unit Circle / Sphere the set of points at distance 1 from the origin

Polar Coordinate System describes a point in the plane with (r, ϑ)

Spherical Coordinate System describes a point in 3-D space with (r, ϑ, φ)

arctan2(y, x) correctly resolves the angle in every quadrant

FURTHER LEARNING

Companion Code & Extended Reading

Code 1 · LA_01_05_01.ipynb

Computing a unit vector and checking for the zero vector

Code 2 · LA_01_05_02.ipynb

Converting between Cartesian and polar coordinates

Code 3 & 4 · LA_01_05_03 / 06.ipynb

Drawing a unit circle and a unit sphere's wireframe

Open-Source Repository

Full code and figure assets: <https://github.com/Visualize-ML>

Q1 – Q10

Q1 Annotate the contour-plot example line by line, and learn to draw contour plots (matplotlib contour_demo).

Q3 Write Python code to plot sine and cosine curves.

Q5 Compute the unit vector of $[4, 4]^T$, and write $[4, 4]^T$ as "length $\times$ direction vector".

Q7 Convert the polar coordinates $(2, \pi/3)$ to Cartesian coordinates.

Q9 Write `unit_vector(a)` returning the unit vector of any nonzero n-D vector $\mathbf{a}$, and verify its length is 1 with `np.linalg.norm()`.

Q2 Learn to draw filled contour plots (matplotlib.pyplot.contourf).

Q4 Convert the Cartesian point $(4, 4)$ to polar coordinates.

Q6 Compute the unit vector of $[1, 2, 2]^T$, and write $[1, 2, 2]^T$ as "length $\times$ direction vector".

Q8 Convert the spherical coordinates $(4, \pi/6, \pi/4)$ to 3-D Cartesian coordinates.

Q10 Write code that detects a zero vector and raises an error or warning instead of dividing by zero when normalizing.

Q11 – Q19

Q11 Plot vector $\mathbf{a}=[5,12]^T$ and its normalized direction vector $\hat{\mathbf{a}}$ with Matplotlib, labeling both coordinates.

Q13 Write `length_direction(a)` returning a vector's length and direction vector, and verify their product equals the original.

Q15 Generate 50 random 2-D vectors of different lengths and plot the concentric-circle pattern their tips form (mimic Figure 4).

Q17 Write `polar_angle(a)` returning the angle α between a 2-D vector and the positive horizontal direction, in both radians and degrees.

Q19 Use `np.arctan2()` to write `cartesian_to_polar(x1, x2)` returning the radial distance r and the polar angle θ in degrees.

Q12 Generate 20 random nonzero 2-D vectors, compute their unit vectors, and plot them — verify every tip lands on the unit circle (mimic Figure 2).

Q14 Write code that decides whether a 2-D vector's tip is inside, on, or outside the unit circle (mimic Figure 3).

Q16 For $\mathbf{a}=[3,4]^T$, compute its scalar projections onto the horizontal and vertical axes, and verify the sum of squares equals $\|\mathbf{a}\|^2$.

Q18 Use Matplotlib to draw the right-triangle diagrams for any two of the six trig functions in Figure 6.

Q20 – Q28

Q20 Use `np.cos()/np.sin()` to write `polar_to_cartesian(r, theta)`, converting polar coordinates back to Cartesian.

Q22 Write `unit_vector_3d(a)` returning the unit vector of a nonzero 3-D vector, and verify its tip lies on the unit sphere.

Q24 Generate 100 random 3-D unit vectors (tips on the unit sphere) and visualize them with a 3-D scatter plot (mimic Figure 9, right panel).

Q26 Write `cartesian_to_spherical(x1, x2, x3)` converting 3-D Cartesian coordinates to spherical coordinates.

Q28 Modify the radius in Code 4 to draw a radius-2 sphere's wireframe with `plot_wireframe()`, and compare it with the unit sphere.

Q21 Use Matplotlib to draw 5 concentric circles of different radii (mimic Figure 8(b)), with angle ticks labeled.

Q23 Use `mpl_toolkits.mplot3d` to plot the 3-D vector $\mathbf{b}=[2,3,6]^T$ and its normalized direction vector.

Q25 Write `spherical_to_cartesian(r, theta, phi)` converting spherical coordinates to 3-D Cartesian coordinates.

Q27 Use Matplotlib to plot the unit vectors of the red, green, and blue basis vectors in RGB space (mimic Figure 10), with a legend.

10 Prompts to Explore Further (1/2)

1

Explain why a unit vector keeps only "direction" information and discards "magnitude" — how does this compare to feature normalization in machine learning?

2

Prove that for any nonzero vector $\mathbf{a}$, its unit vector $\hat{\mathbf{a}}$ points in exactly the same direction as $\mathbf{a}$, with zero angle between them.

3

Use an everyday example to explain "scalar projection" (say, the length of a pole's shadow under the noon sun).

4

What are the respective advantages of polar and Cartesian coordinates? In what situations is polar more natural?

5

Why can the polar angle θ in spherical coordinates be compared to "latitude," and the azimuth ϕ to "longitude"? Where does this analogy break down?

10 Prompts to Explore Further (2/2)

6 Design a small Python experiment verifying that a unit vector still has length 1 after a rotation (invariance under rotation).

7 If every component of a vector is scaled by k , how does its unit vector change? Explain both algebraically and geometrically.

8 Explain the analogy between 3-D unit vectors and unit vectors in RGB color space — where does this analogy have limits?

9 Why must the zero vector be excluded from normalization? How should this edge case be handled gracefully in code?

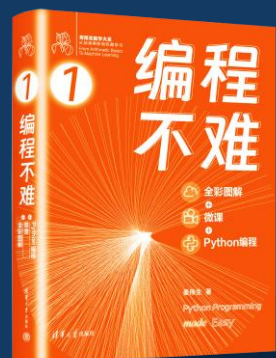
10 Using a concrete scenario (navigation, robot orientation, surface normals in graphics), explain the real-world meaning of a unit vector.

数学

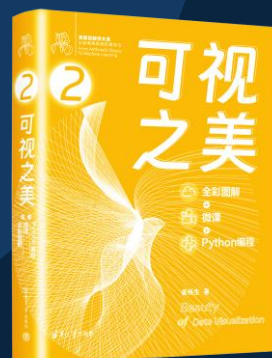
数学基础

线性代数

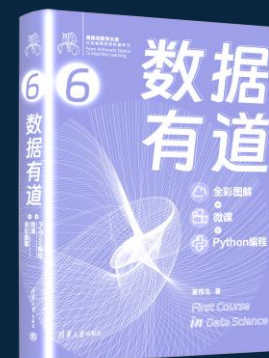
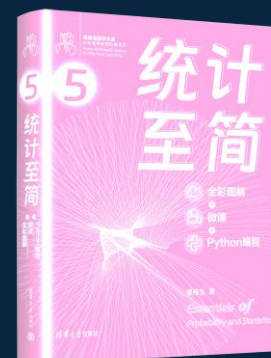
概率统计



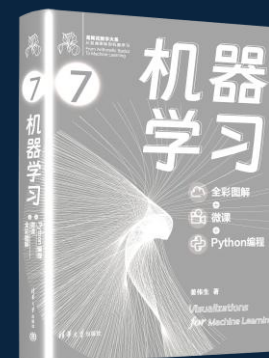
Python编程



数据可视化



数据分析
图和网络



回归、降维
分类、聚类

工具

实践

